De la programación que ya conocemos a Kotlin



Hoy todavía no
Activity · Compose · ViewModel · Navigation · Room

Entorno principal
Kotlin Playground

No volvemos a enseñar programación desde cero.
Traducimos conceptos ya conocidos al lenguaje que usará PMDM.

Qué es Kotlin y dónde encaja

lenguaje de programación

open source

tipado estático

desarrollado por JetBrains

OO + funcional

Kotlin

Primera versión estable 1.0 → 2016

```
archivo fuente: MiPrograma.kt
```

Idea clave
Kotlin es el lenguaje. Android es una plataforma.

JVM

Android

JavaScript

WebAssembly

Native

Kotlin en PMDM: razones y costes

Android

Kotlin es el lenguaje recomendado para nuevas apps Android.
Las APIs de Compose aprovechan funciones, lambdas y extensiones.

JVM + Java

Kotlin/JVM y Java pueden convivir.
Interoperabilidad ≠ mismo lenguaje ni conversión perfecta.

Kotlin Multiplatform

Puede compartir código entre plataformas.
En esta evaluación NO usamos KMP como stack.

Aporta

- concisión
- null safety
- inferencia de tipos
- funciones como valores
- tooling Android Studio

Trade-offs

- nuevos idioms
- concisión excesiva
- interop Java puede debilitar garantías
- compatibilidad de versiones

No confundir

val ≠ objeto inmutable
Kotlin ≠ siempre más rápido
null safety ≠ NPE imposible

PMDM evaluación 1: Android nativo + Jetpack Compose

Primer programa y estructura de un .kt


```
fun main() {
    println("Hola, Kotlin")
}
```

fun → declara una función

main() → punto de entrada

println(...) → llamada

{ ... } → bloque

sin ; normalmente

```
package ejemplo

import kotlin.math.max

// comentario

fun main() {
    println(max(4, 7))
}
```

En Kotlin Playground empezamos sin package ni import cuando no hagan falta.

val, var, tipos e inferencia


```
val nombre = "Ane"
var edad = 20

edad = 21
```

val
no puede reasignarse

var
puede recibir otro valor

```
val intentos = 3
// Kotlin deduce Int
```

Inferencia
el tipo existe aunque no lo escribamos

```
val activo: Boolean = true
val precio: Double = 12.5
val ciudad: String = "Bilbao"
```

Int

Double

Boolean

String

Regla práctica: preferir val cuando no necesitamos reasignar.

No confundir: val controla reasignación; la mutabilidad del objeto se verá con listas.

Operadores, expresiones y Strings

```
a + b
a - b
a * b
a / b
a % b
```

```
a > b
a >= b
a < b
a <= b
a == b
a != b
```

```
activo && edad >= 18
esAdmin || esProfesor
!terminado
```

```
contador += 1
contador++
```

Precisión numérica
5 / 2 → 2
5.0 / 2 → 2.5

```
val nombre = "Ane"
val edad = 20

println("Hola, $nombre")
println("El año que viene: ${edad + 1}")
```

$variable
expresión simple

${expresión}
cálculo o acceso

Decidir: if y when


```
if (edad >= 18) {
    println("Mayor de edad")
} else {
    println("Menor de edad")
}
```

```
val mensaje =
    if (edad >= 18) {
        "Adulto"
    } else {
        "Menor"
    }
```

```
when (opcion) {
    1 -> println("Alta")
    2 -> println("Baja")
    else -> println("Otra")
}
```

```
when {
    nota < 5 -> println("Suspenso")
    nota < 7 -> println("Aprobado")
    else -> println("Notable o más")
}
```

Elegimos la estructura que haga más clara la decisión.

Prueba
Frío si x < 15 / templado si 15 =< x < 25 / calor 25 =< x
primero con if, luego con when.

Repetir: for, rangos y while


```
for (numero in 1..5) {
    println(numero)
}
```

1..5
rango del 1 al 5

```
var intentos = 0

while (intentos < 3) {
    println(intentos)
    intentos++
}
```

```
do {
    // ...
} while (condicion)
```

En nuestras apps usaremos sobre todo for para recorrer datos.

```
var suma = 0

for (n in 1..5) {
    suma += n
}

println(suma)
```

Resultado esperado
15

Nullabilidad, conversiones y smart casts


```
val nombre: String = "Ane"
val apodo: String? = null
```

String
no admite null

String?
puede admitir null

```
if (apodo != null) {
    println(apodo.length)
}
```

```
val n1 = texto.toIntOrNull()
val d1 = precio.toDoubleOrNull()
```

```
resultado?.toString()
    ?: "Sin resultado"
```

?.
llamada segura

```
val nombre: String? = "Ane"
// ?. → llamada segura
println(nombre?.length)
```

?:
valor alternativo

```
val nombre: String? = null
// ?: → valor alternativo
val texto = nombre ?: "Sin nombre"
println(texto)
```

!!
evitar como rutina

```
val nombre: String? = "Ane"
// !! → afirmamos que NO es null
println(nombre!!.length)
```

```
fun mostrar(tarea: Tarea?) {
    if (tarea == null) {
        return
    }

    println(tarea.texto)
}
```

Guard clause + smart cast
Si continúa la ejecución, Kotlin sabe que tarea ya no es null.

```
fun mostrarTarea(tarea: Tarea?) {
   if (tarea == null) return println(tarea.texto)
}
```

Listas, mutabilidad, genéricos y acceso []


```
val numeros: List<Int> =
    listOf(4, 7, 10)
```

List<Int> → lista de Int

```
val numeros =
    mutableListOf<Int>()

numeros.add(4)
numeros.add(7)
numeros.remove(4)
numeros.clear()
```

val
la variable no se reasigna

MutableList
el contenido puede cambiar

numeros[0] = 1

<Int>
tipo de los elementos

```
numeros.isEmpty()
numeros.isNotEmpty()

println(numeros[0])
```

```
for (numero in numeros) {
    println(numero)
}
```

<T> indica un tipo parametrizado.

También puede aparecer en llamadas: cargar<AppDatabase>()

Funciones, return, Unit y argumentos


```
fun sumar(
    a: Int,
    b: Int
): Int {
    return a + b
}
```

parámetros

tipo de retorno

return

```
fun mostrar(texto: String) {
    println(texto)
}
// retorno: Unit
```

```
fun saludar(
    nombre: String,
    saludo: String = "Hola"
): String {
    return "$saludo, $nombre"
}
```

```
saludar(
    nombre = "Ane",
    saludo = "Egun on"
)
```

```
fun doble(n: Int) = n * 2
```

```
fun procesar(dato: String?) {
    if (dato == null) {
        return
    }

    println(dato.length)
}
```

return temprano: puede salir de una función Unit y preparar smart cast

Clases, objetos y data class


```
class Persona(
    val nombre: String,
    var edad: Int
) {
    fun cumplirAnios() {
        edad++
    }

    fun presentarse(): String {
        return "Hola, soy $nombre"
    }
}
```

```
val persona =
    Persona("Ane", 20)

persona.cumplirAnios()
println(persona.edad)
println(persona.presentarse())
```

sin new

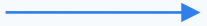
acceso con .

```
data class Producto(
    val nombre: String,
    val precio: Double,
    val disponible: Boolean = true
)
```

data class
clase orientada principalmente a representar datos
Kotlin genera automáticamente varios miembros útiles

Lambdas: código que podemos pasar


```
fun cuadrado(n: Int): Int {
    return n * n
}
```



```
val cuadrado = { n: Int ->
    n * n
}
```

Una expresión lambda crea un valor de tipo función.

```
{ numero ->
    println(numero)
}
```

```
val numeros = listOf(4, 7, 12)

val encontrado =
    numeros.firstOrNull {
        it > 10
    }
```

it → parámetro implícito cuando hay uno

```
hacerAlgo {
    println("Ejecutar después")
}
```

Trailing lambda
Si el último argumento es una lambda, puede escribirse fuera de los paréntesis.

Tipos función, callbacks y referencias ::

() -> Unit
función sin parámetros
sin resultado significativo

(Int) -> Unit
función que recibe Int

(Int) -> String
recibe Int y devuelve String

```
fun procesar(
    numero: Int,
    onTerminar: (Int) -> Unit
) {
    val resultado = numero * 2
    onTerminar(resultado)
}
```

Callback:
función entregada para que otro código la invoque después

```
fun mostrar(n: Int) {
    println(n)
}

procesar(
    4,
    onTerminar = ::mostrar
)
```

::mostrar
referencia a una función existente
no la ejecuta en ese momento

```
viewModel::insertar
```

private, get(), private set e init

```
class Contador {
    private var _valor = 0

    val valor: Int
        get() = _valor

    var resultado: Int? = null
        private set

    init {
        println("Contador creado")
    }

    fun incrementar() {
        _valor++
        resultado = _valor
    }
}
```

private
acceso restringido

get() = ...
getter personalizado

private set
se puede leer desde fuera, pero no asignar

init { ... }
se ejecuta durante la inicialización

```
val c = Contador()
c.incrementar()
println(c.resultado)

c.resultado = 100  // ✕
```

Herencia, override, super, interface y abstract


```
open class Animal {
    open fun sonido() {
        println("...")
    }
}

class Perro : Animal() {
    override fun sonido() {
        super.sonido()
        println("Guau")
    }
}
```

:
hereda o implementa según el contexto

override
redefine un miembro

super
accede a la superclase

Precisión: las clases Kotlin son finales por defecto.

```
interface Guardable {
    fun guardar()
}
```

```
abstract class Repositorio {
    abstract fun cargar(): List<String>
}
```

```
class MainActivity : ComponentActivity()

interface Reiniciable {
    fun reiniciar()
}

class Contador : Reiniciable {
    override fun reiniciar() {
        println("Contador reiniciado")
    }
}

abstract class AppDatabase : RoomDatabase()
```

Sintaxis Kotlin que veremos rodeando frameworks


```
@Composable
fun Pantalla() {
    // ...
}

@Entity(tableName = "tareas")
data class Tarea(...)
```

@...
sintaxis de anotación
el significado depende de la API

```
const val RUTA = "resultado"
```

const val
constante conocida en compilación

```
class Gestor {
    companion object {
        val Factory = ...
    }
}
```

```
Tarea::class
[Tarea::class]

databaseBuilder<AppDatabase>()
```

::class → referencia de clase
[...] → construcción de lista/array solo permita en argumentos de anotaciones/decoraciones
<...>() → tipo indicado en una llamada

```
24.dp
Modifier.padding(24.dp)
```

extensions
reconocer, no crear

by, lazy, this, casts y suspend


```
val mensaje by lazy {
    println("Creando")
    "Hola"
}

var contador by remember {
    mutableStateOf(0)
}
```

by
la propiedad delega parte de su comportamiento
lazy calcula al usarse por primera vez

```
val app = objeto as PmdmApplication
val appSegura = objeto as? PmdmApplication
```

as → cast explícito
as? → cast seguro; puede devolver null

```
this[APPLICATION_KEY]

numeros[0]
```

this
objeto/receptor actual del contexto
[] acceso indexado o por clave

```
suspend fun cargar(): List<Tarea>
```

suspend
marca una función suspendible
No significa «crear un hilo».

Ya podemos avanzar a la próxima presentación


```
package eus.txurdinaga.pmdm.primeraapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PrimeraPantalla()
        }
    }
}

@Composable
fun PrimeraPantalla() {
    Text("Hola PMDM")
}
```

Kotlin que YA conocemos

package · import · class · : SuperClase()
override · fun · parámetro + tipo
Bundle? · super · llamadas · { ... }
lambda / trailing lambda · sintaxis @...

Lo nuevo de la próxima presentación

¿Qué es ComponentActivity?
¿Cuándo se ejecuta onCreate()?
¿Qué hace setContent?
¿Qué significa realmente @Composable?
¿Qué hace Text?

Esta presentación resuelve el lenguaje.
La próxima presentación empieza Android.